

Java Features

Java 8 to 17 migration for Developers

TABLE OF CONTENTS

1. Lambda Expression
2. Stream API
3. Method Reference
4. Default Methods
5. Try with Resources
6. Local Variable Type Inference
7. Switch Expression
8. Text Block
9. Pattern matching for instanceof
10. Record
11. Sealed class

Lambda Expression

(int arg1, String arg2) -> {System.out.println("Two arguments "+arg1+" and "+arg2);}

Argument List Arrow token Body of lambda expression

- 1.**Argument-list:** It can be empty or non-empty as well.
- 2.**Arrow-token:** It is used to link arguments-list and body of expression.
- 3.**Body:** It contains expressions and statements for the lambda expression.

```
double getPiValue() {  
    return 3.1415;  
}
```

Without Lambda Expression



```
() -> 3.1415
```

With Lambda Expression

```
// Using lambda expression to print even elements
// of arrL
arrL.forEach(n -> {
    if (n % 2 == 0)
        System.out.println(n);
});
```

- It is not mandatory to use parentheses if the type of that variable can be inferred from the context.
- Body can be either an expression or a {...} block with statements. The expression (other than a method or constructor call) is simply returned, i.e. () -> 2 is equivalent to () -> {return 2;}

```
// Java program to demonstrate lambda expressions to  
// implement a user defined functional interface.
```

```
@FunctionalInterface
```

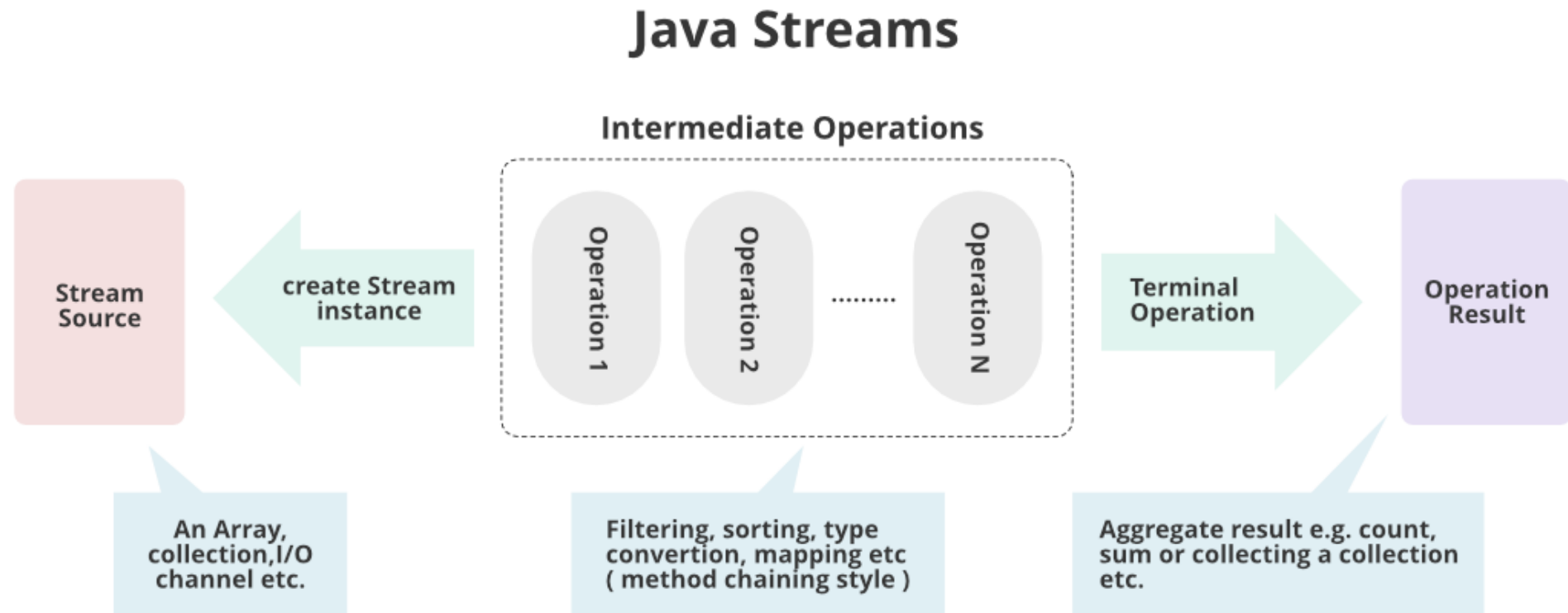
```
interface Square {  
    int calculate(int x);  
}
```

```
class Test {  
    public static void main(String args[])  
    {  
        int a = 5;  
  
        // lambda expression to define the calculate method  
        Square s = (int x) -> x * x;  
  
        // parameter passed and return type must be  
        // same as defined in the prototype  
        int ans = s.calculate(a);  
        System.out.println(ans);  
    }  
}
```

Functional Interfaces

- A **functional interface** is an interface that contains only one abstract method. They can have only one functionality to exhibit.
- *@FunctionalInterface is not implemented or written with a function interface, more than one abstract method can be declared inside it.*

The Stream API provides a way to work with sequences of data in a functional and declarative style, including filtering, mapping, and reducing operations.



Various core operations over Streams.

Core Stream Operations

Intermediate operations return the stream itself so you can chain multiple method calls in a row

Intermediate Operations

- ✓ filter()
- ✓ map()
- ✓ sorted()

Terminal operations return a result of a certain type instead of again a Stream

Terminal Operations

- ✓ forEach()
- ✓ collect()
- ✓ match()
- ✓ count()
- ✓ reduce()

Though, stream operations are performed on all elements inside a collection satisfying a predicate, it is often desired to break the operation whenever a matching element is encountered during iteration. In external iteration, there are certain methods you can use for this purpose

Short-circuit Operations

- ✓ anyMatch()
- ✓ findFirst()

```
public class LambdaExpressions {  
    public static List<Car> findCarsOldWay(List<Car> cars) {  
        List<Car> selectedCars = new ArrayList<>();  
        for (Car car : cars) {  
            if (car.kilometers < 50000) {  
                selectedCars.add(car);  
            }  
        }  
        return selectedCars;  
    }  
}
```

Before Lambda Expressions

Using a Stream and a Lambda Expression

```
public class LambdaExpressions {  
    public static List<Car> findCarsUsingLambda(List<Car> cars) {  
        return cars.stream().filter(car -> car.kilometers < 50000)  
            .collect(Collectors.toList());  
    }  
}
```


Method Reference

A method reference allows us to call functions in classes using a special kind of syntax ::

```
public class MethodReference {  
    List<String> withoutMethodReference =  
        cars.stream().map(car -> car.toString())  
            .collect(Collectors.toList());  
}
```

Using a lambda expression to call the toString() method on each car.

```
public class MethodReference {  
    List<String> methodReference = cars.stream().map(Car::toString)  
        .collect(Collectors.toList());  
}
```

But now we call the toString() method by method reference.

Types of Method References

- **Static Method Reference.**
 - Shorthand for invoking a static method, e.g., `ClassName::staticMethodName`.
- **Instance Method Reference of a particular object.**
 - Concise notation for invoking an instance method on a specific object, e.g., `instance::instanceMethodName`.
- **Instance Method Reference of an arbitrary object of a particular type.**
 - Instance method reference to an arbitrary object in Java is a concise notation for invoking a method on an unspecified instance of a class, represented as `ClassName::instanceMethodName`
- **Constructor Reference.**
 - Compact representation for creating new instances by referring to a constructor, e.g., `ClassName::new`

Default Methods

The default methods were introduced to provide backward compatibility so that existing interfaces can use the lambda expressions without implementing the methods in the implementation class. Default methods are also known as defender methods or virtual extension methods.

```
public class DefaultMethods {  
  
    public interface Logging {  
        void log(String message);  
  
        default void log(String message, Date date) {  
            System.out.println(date.toString() + ": " + message);  
        }  
    }  
}
```

Putting the default keyword allows us to add the implementation of the method inside the interface. Now, our implementation class does not fail with a compiler error even though we didn't implement this new method inside of it.

Try with Resources

Try-with-resources statement is a try statement that declares one or more resources in it. A resource is an object that must be closed once your program is done using it. For example, a File resource or a Socket connection resource.

```
try(declare resources here) {  
    // use resources  
}  
catch(FileNotFoundException e) {  
    // exception handling  
}
```

EXAMPLE

```
// Java Program for try-with-resources
// having single resource

import java.io.*;

class trywithResource {

    public static void main(String[] args)
    {
        try (

            // Creating an object of FileOutputStream
            // to write stream or raw data

            // Adding resource
            FileOutputStream fos= new FileOutputStream("gfgtextfile.txt")) {

            // Custom string input
            String text= "Hello World. This is my java program";
            byte arr[] = text.getBytes();

            fos.write(arr);

        }

        // Catch block to handle exceptions
        catch (Exception e) {

            // Display message for the occurred exception
            System.out.println(e);

        }

        // Display message for successful execution of
        // program
        System.out.println(
            "Resource are closed and message has been written into the gfgtextfile.txt");

    }
}
```

Local Variable Type Inference(LVTI)

- Type inference refers to the automatic detection of the datatype of a variable, done generally at the compiler time.
- Local variable type inference is a feature enabling developers to omit explicit type declarations for local variables (defined within method bodies, initialization blocks, loops, and other blocks such as if-else). The JDK automatically infers the variable type, leaving it to the compiler to determine the datatype.
- Instead of mentioning the variable datatype on the left-side, before the variable, LVTI allows you to simply put the keyword '**var**'.
- Error cases:
 - Not permitted in class fields
 - Not permitted for uninitialized local variables
 - Not allowed as parameter for any methods
 - Not permitted in method return type
 - Not permitted with variable initialized with 'NULL'


```
// Java code for Normal local
// variable declaration
import java.util.ArrayList;
import java.util.List;
class A {
    public static void main(String ap[])
    {
        List<Map> data = new ArrayList<>();
    }
}
```

re-written as:

```
// Java code for local variable
// declaration using LVTI
import java.util.ArrayList;
import java.util.List;
class A {
    public static void main(String ap[])
    {
        var data = new ArrayList<>();
    }
}
```

Switch Expression

- An enhancement to existing switch statement
- Available as preview since Java 12 and standard since Java 14

Switch Expression

- "->" instead of colon (":")
- Comma (",") delimiter for multiple conditions in a case
- "yield" keyword
- All possible cases have to be covered in a switch expression

```
private static String getDaysInMonths(Month month) {  
    var days = switch (month) {  
        case JANUARY, MARCH, MAY, JULY, AUGUST, OCTOBER, DECEMBER -> { yield "31 days"; }  
        case FEBRUARY -> { yield "28 days"; }  
        default -> { yield "30 days"; }  
    };  
    return days;  
}
```

```
private static String getDaysInMonths(Month month) {  
    var days = switch (month) {  
        case JANUARY, MARCH, MAY, JULY, AUGUST, OCTOBER, DECEMBER -> "31 days";  
        case FEBRUARY -> "28 days";  
        default -> "30 days";  
    };  
    return days;  
}
```

Switch Expression

- Can use return statement in switch statement, but not in switch expression

```
private static String getDaysInMonth(Month month) {  
    switch (month) {  
        case JANUARY, MARCH, MAY, JULY, AUGUST, OCTOBER, DECEMBER -> {  
            return "31 days";  
        }  
        case FEBRUARY -> {  
            return "28 days";  
        }  
        default -> {  
            return "30 days";  
        }  
    }  
}
```

Text Block

- New way to write string literals
- Available as preview since Java 13 and released with Java 15
- Has to be started with `"""` (3 double quotes) followed by optional whitespaces and a newline, and ends with another `"""` (3 double quotes)
- Indentation is based on the spaces from the first line

```
String htmlBlock = """
    <html>
        <body>
            <span>example text</span>
        </body>
    </html>""";
```

```
String expected = "<html>\n\n    <body>\n        <span>example text</span>\n    </body>\n</html>";
```

Text Block

- No need to escape double quotes

```
public String getTextWithEscapes() {  
    return ""  
        "fun" with  
        whitespace  
        and other escapes \""  
        "";  
}
```

Text Block

- No need to escape new lines

```
public String getTextWithCarriageReturns() {  
    return ""  
    separated with\r  
    carriage returns"";  
}  
  
@Test  
void givenATextWithCarriageReturns_thenItContainsBoth() {  
    assertThat(subject.getTextWithCarriageReturns())  
        .isEqualTo("separated with\r\ncarriage returns");  
}
```

Text Block

- We can escape new lines by using "\"

```
public String getIgnoredNewLines() {  
    return ""  
        This is a long test which looks to \  
        have a newline but actually does not"";  
}
```

```
@Test  
void givenAStringWithEscapedNewLines_thenTheResultHasNoNewLines() {  
    String expected = "This is a long test which looks to have a newline but actually does not";  
    assertThat(subject.getIgnoredNewLines())  
        .isEqualTo(expected);  
}
```


Text Block

- Ignores all trailing spaces in each line
- We can preserve the trailing spaces using "\s"

```
public String getEscapedSpaces() {  
    return ""  
        line 1.....  
        line 2.....\s  
        "";  
}  
  
@Test  
void givenAStringWithEscapesSpaces_thenTheResultHasLinesEndingWithSpaces() {  
    String expected = "line 1\nline 2      \n";  
    assertThat(subject.getEscapedSpaces())  
        .isEqualTo(expected);  
}
```

Text Block

- We can pass parameters to string using %s

```
String formattedText = ""  
    Some parameter: %s %s  
    "".formatted(testParam1, testParam2);
```

Pattern matching for instanceOf

- Traditional instanceOf

```
if (animal instanceof Cat) {  
    Cat cat = (Cat) animal;  
    cat.meow();  
    // other cat operations  
} else if (animal instanceof Dog) {  
    Dog dog = (Dog) animal;  
    dog.woof();  
    // other dog operations  
}
```

Pattern matching for instanceOf

- Enhanced instanceOf

```
if (animal instanceof Cat cat) {  
    cat.meow();  
} else if (animal instanceof Dog dog) {  
    dog woof();  
}
```

Record

- A simpler way to define a DTO
- Introduced in Java 14

```
public record Person (String name, String address) {}
```

Record

- Constructor with all arguments will be created automatically

```
public Person(String name, String address) {  
    this.name = name;  
    this.address = address;  
}
```

```
Person person = new Person("John Doe", "100 Linda Ln.");
```

```
public record Person(String name, String address) {  
    public Person {  
        Objects.requireNonNull(name);  
        Objects.requireNonNull(address);  
    }  
}
```

```
public record Person(String name, String address) {  
    public Person(String name) {  
        this(name, "Unknown");  
    }  
}
```

Record

- Getters are automatically generated for the fields
- Equals, hashCode, and toString methods are also generated

```
@Test
public void givenValidNameAndAddress_whenGetNameAndAddress_thenExpectedValuesReturned() {
    String name = "John Doe";
    String address = "100 Linda Ln.";

    Person person = new Person(name, address);

    assertEquals(name, person.name());
    assertEquals(address, person.address());
}
```

```
Person[name=John Doe, address=100 Linda Ln.]
```

Record

- We can include static method and variables

```
public record Person(String name, String address) {  
    public static String UNKNOWN_ADDRESS = "Unknown";  
}
```

```
public record Person(String name, String address) {  
    public static Person unnamed(String address) {  
        return new Person("Unnamed", address);  
    }  
}
```

```
Person.UNKNOWN_ADDRESS  
Person.unnamed("100 Linda Ln.");
```


Sealed class

- Sealing allows classes and interfaces to define their permitted subtypes
- Introduced in Java 17
- Earlier, Java assumed code reuse is always a goal, every class can be extended by any number of subclasses
- Main motivation of sealed class is to have the possibility of a superclass to be widely accessible but not widely extensible
- New modifiers and clauses: sealed, non-sealed, and permits
- A permitted subclass must define a modifier: final, sealed, or non-sealed
- The permits clause should be defined after any extends or implements clauses

Sealed class

```
public sealed interface Service permits Car, Truck {  
  
    int getMaxServiceIntervalInMonths();  
  
    default int getMaxDistanceBetweenServicesInKilometers() {  
        return 100000;  
    }  
  
}
```

```
public abstract sealed class Vehicle permits Car, Truck {  
  
    protected final String registrationNumber;  
  
    public Vehicle(String registrationNumber) {  
        this.registrationNumber = registrationNumber;  
    }  
  
    public String getRegistrationNumber() {  
        return registrationNumber;  
    }  
  
}
```

Sealed class

```
public final class Truck extends Vehicle implements Service {  
  
    private final int loadCapacity;  
  
    public Truck(int loadCapacity, String registrationNumber) {  
        super(registrationNumber);  
        this.loadCapacity = loadCapacity;  
    }  
  
    public int getLoadCapacity() {  
        return loadCapacity;  
    }  
  
    @Override  
    public int getMaxServiceIntervalInMonths() {  
        return 18;  
    }  
  
}
```

Sealed class

```
public non-sealed class Car extends Vehicle implements Service {  
  
    private final int numberOfSeats;  
  
    public Car(int numberOfSeats, String registrationNumber) {  
        super(registrationNumber);  
        this.numberOfSeats = numberOfSeats;  
    }  
  
    public int getNumberOfSeats() {  
        return numberOfSeats;  
    }  
  
    @Override  
    public int getMaxServiceIntervalInMonths() {  
        return 12;  
    }  
  
}
```

Sealed class

```
if (vehicle instanceof Car) {  
    return ((Car) vehicle).getNumberOfSeats();  
} else if (vehicle instanceof Truck) {  
    return ((Truck) vehicle).getLoadCapacity();  
} else {  
    throw new RuntimeException("Unknown instance of Vehicle");  
}
```

```
if (vehicle instanceof Car car) {  
    return car.getNumberOfSeats();  
} else if (vehicle instanceof Truck truck) {  
    return truck.getLoadCapacity();  
} else {  
    throw new RuntimeException("Unknown instance of Vehicle");  
}
```

Sealed class

- Supported by Reflection API; two public methods have been added to `java.lang.Class`:
`isSealed()` & `getPermittedSubclasses()`

```
Assertions.assertThat(truck.getClass().isSealed()).isEqualTo(false);  
Assertions.assertThat(truck.getClass().getSuperclass().isSealed()).isEqualTo(true);  
Assertions.assertThat(truck.getClass().getSuperclass().getPermittedSubclasses())  
    .contains(Class.forName(truck.getClass().getCanonicalName()));
```

Sealed class

- Works very well with records

```
public sealed interface Vehicle permits Car, Truck {  
  
    String getRegistrationNumber();  
  
}
```

```
public record Car(int numberOfSeats, String registrationNumber) implements Vehicle {  
  
    @Override  
    public String getRegistrationNumber() {  
        return registrationNumber;  
    }  
  
    public int getNumberOfSeats() {  
        return numberOfSeats;  
    }  
  
}  
  
public record Truck(int loadCapacity, String registrationNumber) implements Vehicle {  
  
    @Override  
    public String getRegistrationNumber() {  
        return registrationNumber;  
    }  
  
    public int getLoadCapacity() {  
        return loadCapacity;  
    }  
  
}
```


A photograph of three business professionals in a modern office setting. A woman on the left and a man in the center are looking towards a man on the right who is shaking hands with the man in the center. A dark blue rectangular overlay with a white border and a blue L-shaped graphic element is positioned on the left side of the image, containing the text 'Thank You'.

Thank You